# The for Loop

Mastering Repetition with Counters in C Programming

for Loop | Nested Loops | Iterations

# 🔁 The for Loop

🔁 The for statement makes it more convenient to **count iterations** of a loop and works well where the **number of iterations is known** before the loop is entered.

### Syntax:

```
for (initialization; test condition; increment or decrement) {
    Statement(s);
}
```

💡 **Main Purpose:** Repeat statement while condition remains true, like while loop, but with additional places for initialization and increment/decrement of control variable.

# How the for Loop Works

🔄 The for loop works in the following manner:

1. **Initialization**
   Executed only once. Generally sets initial value for a counter variable.

2. **Condition Check**
   Condition is checked. If true, loop continues; otherwise, loop finishes.

3. **Statement Execution**
   Statement(s) inside the loop are executed.

4. **Increment/Decrement**
   Whatever is specified in increment/decrement field is executed and loop returns to step 2.

# ✨ Features of the for Loop

### Basic Features

- ✓ More concise
- ✓ Easy to use
- ✓ Highly flexible

### Advanced Features

- ✓ More than one variable can be initialized
- ✓ More than one increment can be applied
- ✓ More than two conditions can be used

💡 **Key Advantage:** The for loop is specially designed to perform a repetitive action with a counter.

# 💻 for Loop Example

```
#include<stdio.h>
#include<conio.h>
void main() {
    int i;
    clrscr();
    for(i = 0; i < 5; i++) {
        printf("MCMT!\t"); // 5 times
    }
    getch();
}
```

**Output:**

```
MCMT  MCMT  MCMT  MCMT  MCMT
```

🔍 **Explanation:** The loop starts with i = 0 and continues as long as i < 5. In each iteration, it prints "MCMT!" and increments i by 1. The loop runs exactly 5 times.

# 🔍 More About for Loop

💡 The for loop in C has several capabilities that are not found in other loop constructs.

## Multiple Variable Initialization

More than one variable can be initialized at a time in the for statement.

```
p = 1;
for (n = 0; n < 17; ++n)
```

Can be rewritten as:

```
for (p = 1, n = 0; n < 17; ++n)
```

## Multiple Increments

The increment section may also have more than one part.

```
for (n = 1, m = 50; n <= m; n = n+1, m = m-j) {
    p = m/n;
    printf("%d %d %d\n", n, m, p);
}
```

# Nested Loops

**Loops within loops** are called nested loops. They are used when multiple conditions need to be tested or multiple sets of iterations are required.

## Nested while

Used when multiple conditions are to be tested.

```
while (condition 1) {
    // statements
    while (condition 2) {
        // statements
        while (condition n) {
            // statements
        }
    }
}
```

## Nested for

Used when multiple set of iterations are required.

```
for ( ; ; ) {
    // statements
    for ( ; ; ) {
        // statements
        for ( ; ; ) {
            // statements
        }
    }
}
```

**Common Use Case:** Generating patterns, matrices, and multi-dimensional data structures.

# 💻 Nested Loop Example

```
#include<stdio.h>
#include<conio.h>
main() {
    int i, N;
    clrscr();
    for(i = 1; i <= 5; i++) {
        for(N = 1; N <= 5; N++) {
            printf("%d ", N);
        }
        printf("\n");
    }
}
```

**Output:**

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

🔍 **Explanation:** The outer loop runs 5 times (for each row), and the inner loop runs 5 times for each iteration of the outer loop (for each column). This creates a 5x5 grid of

numbers.

# 🎯 Summary

### 🔁 for Loop

Ideal for known number of iterations with initialization, condition, and increment in one statement

### 🔁 Nested Loops

Loops within loops for complex patterns and multi-dimensional operations

### ✨ Advanced Features

Multiple initializations and increments in for loops for complex control

🚀 **Master these loop structures to create efficient and powerful C programs!**